

FORMAL METHODS

Hoare logic

- Hoare Logic provides a structured method for verifying program correctness by using logical assertions and rules. It is applicable to assignments, conditional statements, and loops, making it a powerful tool for formal reasoning in software engineering.

Hoare logic

Partial correctness triples

Hoare logic uses **partial correctness triples** (also “Hoare triples”) for specifying and reasoning about the behaviour of programs:

$$\{P\} \ C \ \{Q\}$$

is a logical statement about a command C ,
where P and Q are state predicates:

- P is called the precondition, and describes the initial state;
- Q is called the postcondition, and describes the terminal state.



Components of Hoare Logic

•
Assertions: Logical conditions about the program state.

Hoare Triples: Describe the behavior of code.

Weakest Precondition (WP): The weakest assertion that guarantees a postcondition.



$$\{P\} \ C \ \{Q\}$$

State predicates P and Q can refer to program variables from C and will be written using standard mathematical notations together with **logical operators** like:

- $\wedge$ (“and”), $\vee$ (“or”), $\neg$ (“not”) and $\Rightarrow$ (“implies”)

For instance, the predicate $X = Y + 1 \wedge Y > 0$ describes states in which the variable Y contains a positive value and the value of X is equal to the value of Y plus 1.

The partial correctness triple $\{P\} C \{Q\}$ holds if and only if:

- whenever C is executed in an initial state satisfying P
- and this execution terminates
- then the terminal state of the execution satisfies Q .

For instance,

- $\{X = 1\} X := X + 1 \{X = 2\}$ holds
- $\{X = 1\} X := X + 1 \{X = 3\}$ does not hold

Hoare Logic is a formal system used for reasoning about the correctness of computer programs. It is based on the use of **Hoare triples** of the form:

$$\{P\} S \{Q\}$$

Where:

- P (Precondition): A logical condition that must be true before executing the program statement S .
- S (Program Statement): The program or code fragment being executed.
- Q (Postcondition): A logical condition that must be true after executing S , assuming P was true initially.

Basic Rules of Hoare Logic

Assignment Rule:

$$\{P[x \mapsto e]\} x := e \{P\}$$

The precondition must hold after substituting e for x in P .

Sequential Composition Rule:

$$\{P\} S_1 \{R\}, \{R\} S_2 \{Q\} \quad \text{implies} \quad \{P\} S_1; S_2 \{Q\}$$

If S_1 establishes R from P , and S_2 establishes Q from R , the entire sequence is correct.

- Conditional Rule (If-Else):

$$\frac{\{P \wedge B\} S_1 \{Q\}, \{P \wedge \neg B\} S_2 \{Q\}}{\{P\} \text{ if } B \text{ then } S_1 \text{ else } S_2 \{Q\}}$$

If the condition B holds, execute S_1 ; otherwise, execute S_2 .

While Loop Rule:

$$\frac{\{P \wedge B\} S \{P\}}{\{P\} \text{ while } B \text{ do } S \{P \wedge \neg B\}}$$

Here, P is an invariant, and the loop is correct if S preserves the invariant.

Examples of Hoare Logic

Example 1: Assignment

We want to verify the Hoare triple:

$$\{x = 2\} x := x + 1 \{x = 3\}$$

- Precondition: $x = 2$
- Postcondition: $x = 3$

Using the **assignment rule**, we substitute the right-hand side $x + 1$ for x in the postcondition:

$$x + 1 = 3 \quad \Rightarrow \quad x = 2$$

Thus, the Hoare triple holds.

Example 2: Conditional Statement

```
if ( $x > 0$ ) then  
   $y := x + 1$   
else  
   $y := -x$ 
```

We want to verify the Hoare triple:

$$\{x \geq 0\} \text{ if } (x > 0) \text{ then } y := x + 1 \text{ else } y := -x \{y > 0\}$$

- Precondition: $x \geq 0$
- Postcondition: $y > 0$

We check the two branches separately:

1. **Then-branch** ($x > 0$):

- Precondition: $x > 0$
- Postcondition: $y = x + 1$
- Since $x > 0$, $y > 0$ is satisfied.

2. **Else-branch** ($x \leq 0$):

- Precondition: $x \leq 0$
- Postcondition: $y = -x$
- Since $x \leq 0$, $y \geq 0$, and the original condition $x = 0$ implies $y = 0$, thus $y > 0$ holds.

Hence, the Hoare triple holds for the conditional.

Example 3: While Loop

```
while (x > 0) do  
  x := x - 1
```

We want to prove:

$$\{x \geq 0\} \text{ while } (x > 0) \text{ do } x := x - 1 \{x = 0\}$$

- Precondition: $x \geq 0$
- Postcondition: $x = 0$

We use $x \geq 0$ as the **invariant**, which remains true throughout the loop. Each iteration decreases x , and when $x = 0$, the loop terminates.

The WHILE language

Commands of the WHILE language

WHILE is the prototypical imperative language. Programs consist of commands, which include branching, iteration, and assignment:

$$\begin{array}{lcl} C & ::= & \text{skip} \\ & | & C_1; C_2 \\ & | & X := E \\ & | & \text{if } B \text{ then } C_1 \text{ else } C_2 \\ & | & \text{while } B \text{ do } C \end{array}$$

Here, X is a variable, E is an arithmetic expression, which evaluates to an integer, and B is a boolean expression, which evaluates to a boolean.

States are mappings from variables to integers, $Var \rightarrow \mathbb{Z}$.

Expressions of the WHILE language

The grammar for arithmetic expressions and boolean expressions includes the usual arithmetic operations and comparison operators, respectively:

$$\begin{aligned} E &::= N \mid X \mid E_1 + E_2 && \text{arithmetic expressions} \\ &\mid E_1 - E_2 \mid E_1 \times E_2 \mid \dots \end{aligned}$$
$$\begin{aligned} B &::= \mathbf{T} \mid \mathbf{F} \mid E_1 = E_2 && \text{boolean expressions} \\ &\mid E_1 \leq E_2 \mid E_1 \geq E_2 \mid \dots \end{aligned}$$

Expressions do not have side effects.

Example 7: Sum of First n Numbers

Consider the program that computes the sum of the first n integers:

```
sum := 0;  
while (n > 0) do  
  sum := sum + n;  
  n := n - 1
```

We want to prove:

$$\{n \geq 0\} \dots \{sum = \frac{n(n+1)}{2}\}$$

1. **Invariant:** At each iteration, $sum + \frac{n(n+1)}{2}$ remains constant.
2. After the loop terminates, $n = 0$ and the sum becomes $\frac{n(n+1)}{2}$.

Thus, the program correctly computes the sum.



A **loop invariant** is a logical condition that remains true before and after each iteration of a loop. In the context of Hoare Logic, a loop invariant is crucial for proving the correctness of loops, particularly when using the **while** rule.

Purpose of a Loop Invariant:

- It helps in reasoning about the behavior of a loop.
 - It provides a mechanism to prove that a loop eventually leads to the desired outcome.
 - During each iteration, the invariant holds, meaning it's preserved throughout the execution of the loop.
- 

Structure of the While Rule with Loop Invariant:

The general form of the **while** rule is:

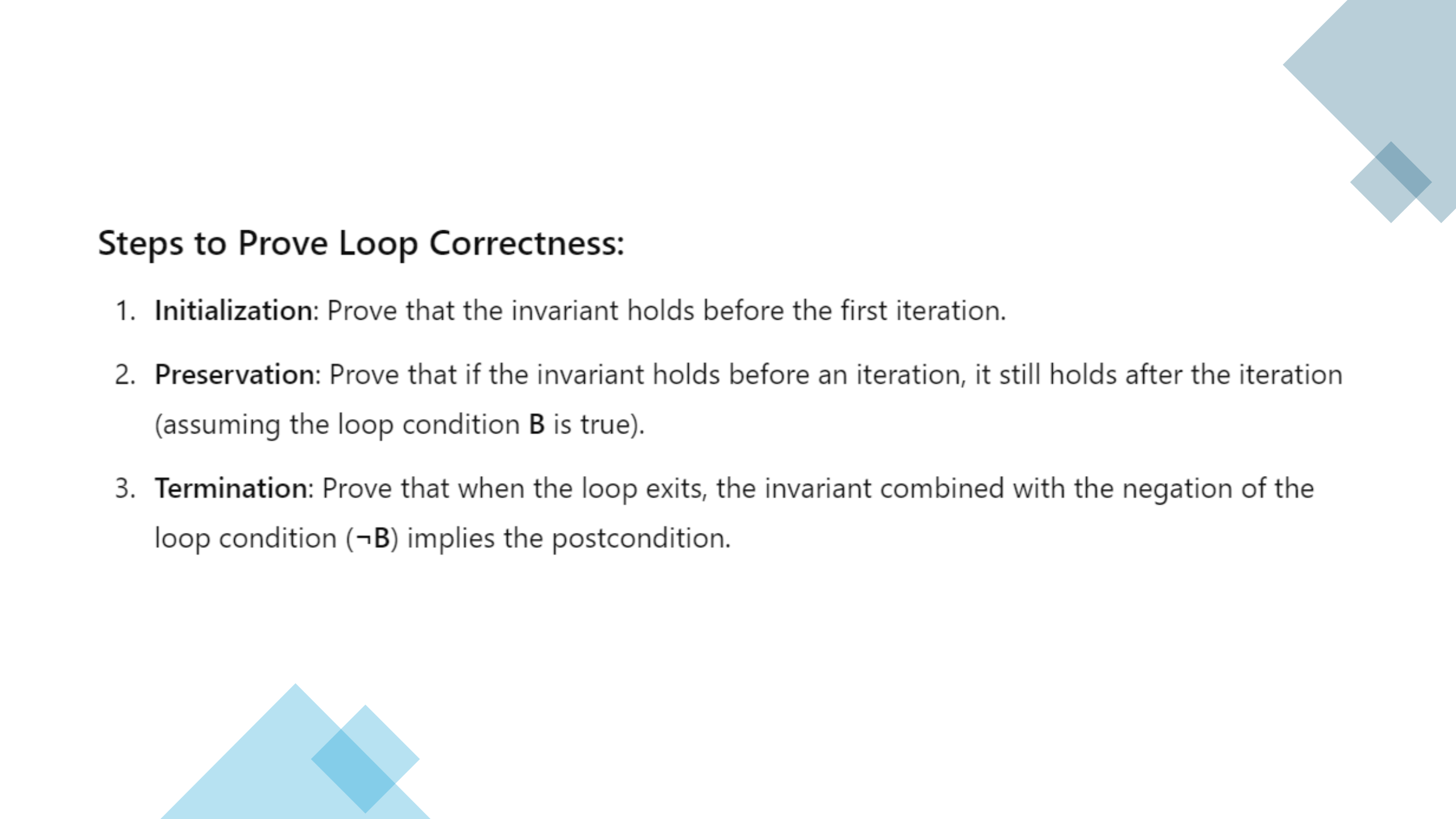
$$\{I \wedge B\} C \{I\}, \quad \Rightarrow \quad \{I\} \text{ while } B \text{ do } C \text{ done } \{I \wedge \neg B\}$$

Where:

- **I** is the loop invariant.
- **B** is the loop condition.
- **C** is the loop body.

The rule means:

1. **I** (the invariant) holds before the loop starts.
2. While the loop condition **B** holds true, each iteration maintains **I**.
3. When the loop terminates, **B** is false, and **I** still holds.



Steps to Prove Loop Correctness:

1. **Initialization:** Prove that the invariant holds before the first iteration.
2. **Preservation:** Prove that if the invariant holds before an iteration, it still holds after the iteration (assuming the loop condition **B** is true).
3. **Termination:** Prove that when the loop exits, the invariant combined with the negation of the loop condition ($\neg \mathbf{B}$) implies the postcondition.



Activity:

```
x = 1;  
y = 0;  
while (x <= 100) {  
    y = y + x;  
    x = x + 1;  
}
```

- i. Specify a loop invariant for the while loop in the form $\{ I \}$ while (condition) $\{ S \}$ $\{ I \}$ where I is the loop invariant.
- i. Using the loop invariant, prove that the program correctly computes the sum of integers from 1 to 100. Show each step of your proof.

Total Correctness Triple:

In Hoare Logic, **total correctness** and **partial correctness** are two distinct ways of reasoning about the correctness of a program.

- A **partial correctness triple** guarantees that if the program terminates, the result will be correct according to the postcondition.
- A **total correctness triple** extends this guarantee by ensuring that the program not only produces the correct result if it terminates, but also that it **will terminate**.

$$[P] \ C \ [Q]$$

$$[P] \ C \ [Q]$$

The total correctness triple $[P] \ C \ [Q]$ holds if and only if:

- whenever C is executed in an initial state satisfying P
- then the execution must terminate
- and the terminal state must satisfy Q .

There is no standard notation for total correctness triples, but we will use $[P] \ C \ [Q]$.

The following total correctness triple does not hold:

$$[X = 1] \text{ while } X > 0 \text{ do } X := X + 1 [X = 0]$$

- the loop never terminates when executed from an initial state where X is positive

The following total correctness triple does hold:

$$[X = 0] \text{ while } X > 0 \text{ do } X := X + 1 [X = 0]$$

- the loop always terminates immediately when executed from an initial state where X is zero

Specifications

$$\{\perp\} C \{Q\}$$

- this says nothing about the behaviour of C , because $\perp$ never holds for any initial state

$$\{\top\} C \{Q\}$$

- this says that whenever C halts, Q holds

$$\{P\} C \{T\}$$

- this holds for every precondition P and command C , because T always holds in the terminate state

$[P] C [T]$

- this says that C always terminates when executed from an initial state satisfying P

$[T] C [Q]$

- this says that C always terminates in a state where Q holds

auxiliary variables

Consider a program C that computes the maximum value of two variables X and Y and stores the result in a variable Z .

Is this a good specification for C ?

$$\{\top\} C \{(X \leq Y \Rightarrow Z = Y) \wedge (Y \leq X \Rightarrow Z = X)\}$$

No! Take C to be $X := 0; Y := 0; Z := 0$, then C satisfies the above specification. The postcondition should refer to the **initial** values of X and Y .

In Hoare logic we use **auxiliary variables** which do not occur in the program to refer to the initial value of variables in postconditions.

For instance, $\{X = x \wedge Y = y\} C \{X = y \wedge Y = x\}$, expresses that if C terminates then it exchanges the values of variables X and Y .

Here x and y are auxiliary variables (or ghost variables) which are not allowed to occur in C and are only used to name the initial values of X and Y .

Informal convention: program variables are uppercase and auxiliary variables are lowercase.

Conclusion:

The primary goal of formal methods in software engineering is to **ensure software correctness and reliability** through rigorous mathematical techniques. This involves:

1. **Verification:** Checking that the software meets its specifications and behaves correctly.
2. **Error Prevention:** Identifying and fixing potential errors early in development.
3. **Formal Proofs:** Providing mathematical proofs that certain properties hold, ensuring high confidence in the software's behavior.
4. **Reliability:** Enhancing the robustness and dependability of the software.
5. **Rigorous Development:** Supporting a structured and disciplined approach to software design and verification.